Reinforcement Learning: An Introduction

Markov Decision Process

A **Markov Decision Process (MDP)** is a 4-tuple: (S, A, P_a, R_a):
S is the **state space**. It is the set of all states, which can be either discrete or continuous.
A is the **action space**. It is the set of all actions, which can be either discrete or continuous.
$p_a(s, s')$ for $a \in A$ and $s, s' \in S$ is the probability of action a in state s at time t leading to state s' at time $t + 1$.
$r_a(s, s')$ for $a \in A$ and $s, s' \in S$ is the **reward** received by taking action a and transition from state s to s'.
A **policy** π maps the state space S to the action space A, or to $P(A)$ (ie. $\pi(s)$ is a distribution over A).

Note that the definition of $p_a(s, s')$ assumes the transition probability is time homogenous and has Markov property. The state transition is determined by the **environment**.

The **agent** (or player) interacts with the environment in the following loop:
(1) At time t, agent is in state s_t.
(2) Agent does action a according to policy π.
(3) Environment updates state s_{t+1} according to $p_a(s_t, s_{t+1})$.
(4) Environment generates a reward $r_a(s_t, s_{t+1})$.
(5) Agent is now in state s_{t+1} and continues the loop from (1).

Note that the randomness in this process comes from
(1) stochastic agent's action;
(2) state updates from the environment.

While looping from (1) to (5) as above, a **(state, action, reward) trajectory** is formed:

$$(s_0, a_0, r_0) \to (s_1, a_1, r_1) \to ... \to (s_{T-1}, a_{T-1}, r_{T-1}).$$

A **return** (or cumulative future reward) form time t is defined as

$$\tilde{U}_t = R_t + R_{t+1} + ... = \sum_{i \geq t} R_i.$$

A **discounted return** with a discount rate γ from time t is

$$U_t = R_t + \gamma R_{t+1} + ... = \sum_{i \geq t} \gamma^{i-t} R_i.$$

Note that U_t is a random variable. Its randomness comes from state transition and actions. In other words, U_t depends on the random variables $S_t, S_{t+1}, ...$ and $A_t, A_{t+1}, ...$

Given a policy π, the **action-value function** is defined as

$$Q_\pi(s_t, a_t) := \mathbb{E}\left[U_t | S_t = s_t, A_t = a_t\right],$$

where the expectation is over all A_i and S_i for $i > t$.
The **optimal action-value function** is

$$Q^*(s_t, a_t) := \max_\pi Q_\pi(s_t, a_t).$$

The **state-value function** is defined as

$$V_\pi(s_t) := \mathbb{E}_{A \sim \pi(\cdot|s_t)}\left[Q_\pi(s_t, A)\right].$$

For a policy π, $Q_\pi(s,a)$ indicates how good an agent picks an action a in state s. $V_\pi(s)$ indicates how good the state s is. $\mathbb{E}_S[V_\pi(S)]$ indicates how good a policy π is.

Value-Based Learning

The optimal action-value function Q^* can guide agent's action by

$$a_t = \arg\max_a Q^*(s_t, a)$$

at state s_t. Value-based learning is to learn the optimal action-value function Q^*. Let $Q(s,a;\theta)$ be a neural network (with parameters θ) approximating the optimal action-value function Q^*. To train $Q(s,a;\theta)$, we use **Q-Learning**, which is a type of **Temporal Difference Learning** (TD Learning).
Recall that $U_t = \sum_{i\geq t} \gamma^{i-t} R_i$. Note

$$\begin{aligned} U_t &= R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + ... \\ &= R_t + \gamma(R_{t+1} + \gamma R_{t+2} + ...) \\ &= R_t + \gamma U_{t+1}. \end{aligned}$$

Hence, at time t,

$$Q(s_t, a_t; \theta) \approx r_t + \gamma \max_a Q(s_{t+1}, a; \theta),$$

where r_t is the reward by taking action a_t and transition from s_t to s_{t+1}. We let the TD target

$$y_t(\theta) := r_t + \gamma \max_a Q(s_{t+1}, a; \theta).$$

The loss function is

$$L_t(\theta) := \frac{1}{2}(Q(s_t, a_t; \theta) - y_t(\theta))^2.$$

The Q-learning algorithm (for one step) therefore is:
(1) Observe state s_t and take action a_t;

(2) Compute $q_t := Q(s_t, a_t; \theta_t)$ and differentiate

$$d_t = \frac{\partial Q(s_t, a_t; \theta)}{\partial \theta}(\theta_t);$$

(3) Environment gives the next state s_{t+1} and reward r_t;
(4) Compute TD target $y_t = r_t + \gamma \max_a Q(s_{t+1}, a; \theta_t)$;
(5) Gradient descent on θ:

$$\theta_{t+1} = \theta_t - \alpha(q_t - y_t)d_t,$$

where α is the learning rate.

Policy-Based Learning

In policy-based learning, we learn a neural network $\pi(a|s;\theta)$ approximating the policy π. For a trajectory

$$\tau : (s_0, a_0, r_0) \to (s_1, a_1, r_1) \to ... \to (s_{T-1}, a_{T-1}, r_{T-1})$$

we let $R(\tau) := \sum_{t=0}^{T-1} \gamma^t r_t$ be its discounted total return. Let $p(\tau|\theta)$ be the probability of observing τ under the policy $\pi(a|s;\theta)$. Note that $p(\tau|\theta)$ also depends on the state-transition probability $p_a(s, s')$, which is completely determined by the environment.
The objective is to maximize the expected return:

$$J(\theta) := \mathbb{E}_{\tau \sim p(\tau|\theta)}[R(\tau)] = \int R(\tau)p(\tau|\theta)\ d\tau.$$

To get the derivative of $J(\theta)$, we have

$$\nabla_\theta J(\theta) = \nabla_\theta \int R(\tau)p(\tau|\theta)\ d\tau = \int R(\tau)\nabla_\theta p(\tau|\theta)\ d\tau.$$

Notice that

$$p(\tau|\theta) = p(s_0)\left(\prod_{t=0}^{T-2}\pi(a_t|s_t;\theta)p_{a_t}(s_t,s_{t+1})\right)\pi(a_{T-1}|s_{T-1};\theta).$$

In this product the probabilities $p(s_0)$ and $p_{a_t}(s_t, s_{t+1})$ are determined by the environment and intractable. One way to bypass this issue is to take logarithm:

$$\nabla_\theta p(\tau|\theta) = p(\tau|\theta)\nabla_\theta \log p(\tau|\theta)$$

and

$$\begin{aligned}\nabla_\theta \log p(\tau|\theta) &= \nabla_\theta \log p(s_0) + \nabla_\theta \sum_{t=0}^{T-2} \log p_{a_t}(s_t, s_{t+1}) + \nabla_\theta \sum_{t=0}^{T-1} \log \pi(a_t|s_t;\theta) \\ &= \sum_{t=0}^{T-1} \nabla_\theta \log \pi(a_t|s_t;\theta).\end{aligned}$$

Hence,

$$\begin{aligned}\nabla_\theta J(\theta) &= \int \left(\sum_{t=0}^{T-1} \nabla_\theta \log \pi(a_t|s_t;\theta)\right) R(\tau)p(\tau|\theta)\ d\tau \\ &= \mathbb{E}_{\tau\sim p(\tau|\theta)}\left[\left(\sum_{t=0}^{T-1} \nabla_\theta \log \pi(a_t|s_t;\theta)\right) R(\tau)\right].\end{aligned}$$

This shows, if we define (the **Reinforce Estimator**)

$$\hat{g}(\tau) := \left(\sum_{t=0}^{T-1} \nabla_\theta \log \pi(a_t|s_t;\theta)\right) R(\tau),$$

then it is an unbiased estimator of the gradient of expected return:

$$\mathbb{E}_{\tau\sim p(\tau|\theta)}\left[\hat{g}(\tau)\right] = \nabla_\theta J(\theta).$$

Hence, the Reinforce algorithm (for one step) is:

(1) Observe a trajectory according to policy $\pi(a|s;\theta)$:

$$\tau : (s_0, a_0, r_0) \to (s_1, a_1, r_1) \to ... \to (s_{T-1}, a_{T-1}, r_{T-1});$$

(2) Compute $\hat{g}(\tau)$;
(3) Gradient ascent on θ: $\theta = \theta + \alpha\hat{g}(\tau)$, where α is the learning rate.

One problem with the Reinforce Estimator is its high variance. We show an upper bound of its variance under some regulatory assumptions. We restrict the scope by let the trajectories all have length T and denote this restricted probability by $p(\tau|\theta;T)$. Assume the rewards are bounded: $r_t \leq r$ for all $0 \leq t \leq T-1$. Also assume

$$\mathbb{E}_{s_t,a_t}\left[||\nabla_\theta \log \pi(a_t|s_t;\theta)||^2\right] \leq C$$

for all t and for some bound C.

Proposition. *Under the assumptions above, the total variance*

$$\mathrm{Var}(\hat{g}) := \mathbb{E}_{\tau\sim p(\tau|\theta;T)}\left[\left|\left|\hat{g}(\tau) - \mathbb{E}_{\tau\sim p(\tau|\theta;T)}\left[\hat{g}(\tau)\right]\right|\right|^2\right]$$

satisfies

$$\mathrm{Var}(\hat{g}) \leq T^3 r^2 C.$$

Proof. For length T trajectories $(s_t, a_t, r_t)_{0\leq t\leq T-1}$, we denote

$$X(a_t, s_t) := \nabla_\theta \log \pi(a_t|s_t;\theta).$$

Then

$$\begin{aligned}\mathrm{Var}(\hat{g}) &= \mathbb{E}_{\tau\sim p(\tau|\theta;T)}\left[||\hat{g}(\tau)||^2\right] - \left|\left|\mathbb{E}_{\tau\sim p(\tau|\theta;T)}\left[\hat{g}(\tau)\right]\right|\right|^2 \\ &\leq \mathbb{E}_{\tau\sim p(\tau|\theta;T)}\left[||\hat{g}(\tau)||^2\right] \\ &= \mathbb{E}_{\tau\sim p(\tau|\theta;T)}\left[R(\tau)^2\left|\left|\sum_{t=0}^{T-1} X(a_t,s_t)\right|\right|^2\right]. \quad (1)\end{aligned}$$

We first note

$$\begin{aligned}\mathbb{E}_{a_t\sim\pi(\cdot|s_t;\theta)}\left[X(a_t,s_t)|s_t\right] &= \int (\nabla_\theta \log \pi(a_t|s_t;\theta))\pi(a_t|s_t;\theta)\ da_t \\ &= \int \nabla_\theta \pi(a_t|s_t;\theta)\ da_t \\ &= \nabla_\theta \int \pi(a_t|s_t;\theta)\ da_t \\ &= \nabla_\theta 1 = 0.\end{aligned}$$

Let $t < k$ and, by the law of total expectation,

$$\mathbb{E}\left[X(a_t,s_t)^\top X(a_k,s_k)\right] = \mathbb{E}\left[\mathbb{E}\left[X(a_t,s_t)^\top X(a_k,s_k)|s_t,a_t,s_k\right]\right].$$

Conditioning on s_t and a_t completely determines $X(a_t,s_t)$, and therefore

$$\begin{aligned}&\mathbb{E}\left[\mathbb{E}\left[X(a_t,s_t)^\top X(a_k,s_k)|s_t,a_t,s_k\right]\right] \\ &= \mathbb{E}\left[X(a_t,s_t)^\top \mathbb{E}\left[X(a_k,s_k)|s_t,a_t,s_k\right]\right] \\ &= \mathbb{E}\left[X(a_t,s_t)^\top \mathbb{E}\left[X(a_k,s_k)|s_k\right]\right] \\ &= \mathbb{E}\left[X(a_t,s_t)^\top 0\right] = 0.\end{aligned}$$

Hence, the bound in (1) is

$$\begin{aligned}
&\mathbb{E}_{\tau\sim p(\tau|\theta;T)}\left[R(\tau)^2\left|\left|\sum_{t=0}^{T-1}X(a_t,s_t)\right|\right|^2\right] \\
&= R(\tau)^2\sum_{t=0}^{T-1}\mathbb{E}_{\tau\sim p(\tau|\theta;T)}\left[||X(a_t,s_t)||^2\right] \\
&\leq (Tr)^2(TC) = T^3r^2C
\end{aligned}$$

by the assumptions.

□

Empirical variance scaling shows the bound $O(T^3)$ is accurate. In other words, $\mathrm{Var}(\hat{g})$ is $\Theta(T^3)$.

Actor-Critic Method

The actor-critic method learns both the policy and the value function through two neural networks. Let $\pi(a|s;\theta)$ be a neural network **(actor)** approximating the policy. Let $Q(s,a;w)$ be a neural network **(critic)** approximating the action-value function.
The objective for the value network is the (squared) TD difference:

$$\delta_t := Q(s_t,a_t;w) - (r_t + \gamma Q(s_{t+1},a_{t+1},w)).$$

The policy network (actor) $\pi(a|s;\theta)$ is used to get a_t (based on s_t) and a_{t+1} (based on s_{t+1}).
The objective for the policy network is the expected return (written in a slightly different way):

$$J(\theta) = \mathbb{E}\left[Q_{\pi(a|s;\theta)}(s,a)\right].$$

In policy-based learning, we derived the reinforce estimator for the derivative of

expected return w.r.t the parameters using trajectories. A similar result is that

$$(\nabla_\theta \log \pi(a|s;\theta))Q_{\pi(a|s;\theta)}(s,a)$$

is also an unbiased estimator of $\nabla_\theta J(\theta)$ in the sense that

$$\nabla_\theta J(\theta) = \mathbb{E}\left[(\nabla_\theta \log \pi(a|s;\theta))Q_{\pi(a|s;\theta)}(s,a)\right],$$

where the expectation is on states s and actions $a \sim \pi(a|s;\theta)$.
The value network (critic) $Q(s,a;w)$ is used to approximate $Q_{\pi(a|s;\theta)}(s,a)$.

The algorithm (for one step) is:
(1) At time t and state s_t, sample a_t based on $\pi(a_t|s_t;\theta_t)$;
(2) Perform a_t to get s_{t+1} and reward r_t;
(3) Sample a_{t+1} based on s_{t+1} and $\pi(a_t|s_t;\theta_t)$;
(4) Policy update:

$$\theta_{t+1} = \theta_t + \alpha_\theta(\nabla_\theta \log \pi(a_t|s_t;\theta_t))Q(s_t,a_t;w_t);$$

(5) Compute the TD difference

$$\delta_t = Q(s_t,a_t;w_t) - (r_t + \gamma Q(s_{t+1},a_{t+1};w_t))$$

and update the value network:

$$w_{t+1} = w_t - \alpha_w \delta_t \nabla_w Q(s_t,a_t;w_t).$$

Actor-Critic Method with Baseline

The vanilla actor-critic method still suffers high variance during policy training because of the gradient estimator

$$(\nabla_\theta \log \pi(a|s;\theta))Q_{\pi(a|s;\theta)}(s,a).$$

One way to reduce the variance is to subtract a baseline $b(s)$ from $Q_{\pi(a|s;\theta)}(s,a)$.

Note that the baseline $b(s)$ only depends on s, so that after subtraction it is still unbiased:

$$\begin{aligned}
&\mathbb{E}\left[(\nabla_\theta \log \pi(a|s;\theta))b(s)\right] \\
&= \mathbb{E}_s\left[b(s)\mathbb{E}_{a\sim\pi(\cdot|a;\theta)}\left[(\nabla_\theta \log \pi(a|s;\theta))\right]\right] \\
&= \mathbb{E}_s\left[b(s)\mathbb{E}_{a\sim\pi(\cdot|a;\theta)}\left[\frac{\nabla_\theta \pi(a|s;\theta)}{\pi(a|s;\theta)}\right]\right] \\
&= \mathbb{E}_s\left[b(s)\sum_a\left[\frac{\nabla_\theta \pi(a|s;\theta)}{\pi(a|s;\theta)}\right]\pi(a|s;\theta)\right] \\
&= \mathbb{E}_s\left[b(s)\nabla_\theta\left(\sum_a \pi(a|s;\theta)\right)\right] \\
&= \mathbb{E}_s\left[b(s)\nabla_\theta(1)\right] = 0
\end{aligned}$$

We then decide a proper baseline. After subtracting the baseline

$$\hat{g}(s,a) := (\nabla_\theta \log \pi(a|s;\theta))(Q_{\pi(a|s;\theta)}(s,a) - b(s)),$$

the total variance is

$$\mathrm{Var}(\hat{g}(s,a)) = \mathbb{E}\left[||\hat{g}(s,a)||^2\right] - ||\mathbb{E}\left[\hat{g}(s,a)\right]||^2.$$

Since $\hat{g}$ is still unbiased,

$$\mathbb{E}\left[\hat{g}(s,a)\right] = \nabla_\theta J(\theta)$$

and therefore we focus on $\mathbb{E}\left[||\hat{g}(s,a)||^2\right]$.

$$\begin{aligned}
\mathbb{E}\left[||\hat{g}(s,a)||^2\right] &= \mathbb{E}\left[||\nabla_\theta \log \pi(a|s;\theta)||^2 (Q_{\pi(a|s;\theta)}(s,a) - b(s))^2\right] \\
&= \mathbb{E}_s\left[\mathbb{E}_{a\sim\pi(\cdot|s;\theta)}\left[||\nabla_\theta \log \pi(a|s;\theta)||^2 (Q_{\pi(a|s;\theta)}(s,a) - b(s))^2\right]\right] \\
&= \mathbb{E}_s\left[\sum_a ||\nabla_\theta \log \pi(a|s;\theta)||^2 (Q_{\pi(a|s;\theta)}(s,a) - b(s))^2 \pi(a|s;\theta)\right].
\end{aligned}$$

Note, for any fixed state s,

$$\sum_a ||\nabla_\theta \log \pi(a|s;\theta)||^2 \left(Q_{\pi(a|s;\theta)}(s,a) - b(s)\right)^2 \pi(a|s;\theta)$$

has a minimizer

$$\begin{aligned} b^*(s) &= \frac{\sum_a ||\nabla_\theta \log \pi(a|s;\theta)||^2 Q_{\pi(a|s;\theta)}(s,a)\pi(a|s;\theta)}{\sum_a ||\nabla_\theta \log \pi(a|s;\theta)||^2 \pi(a|s;\theta)} \\ &= \frac{\mathbb{E}_{a\sim\pi(\cdot|s;\theta)}\left[||\nabla_\theta \log \pi(a|s;\theta)||^2 Q_{\pi(a|s;\theta)}(s,a)\right]}{\mathbb{E}_{a\sim\pi(\cdot|s;\theta)}\left[||\nabla_\theta \log \pi(a|s;\theta)||^2\right]}, \end{aligned}$$

but, for simplicity, we use

$$\begin{aligned} b(s) &= \mathbb{E}_{a\sim\pi(\cdot|s;\theta)}\left[Q_{\pi(a|s;\theta)}(s,a)\right] \\ &= V_{\pi(\cdot|s;\theta)}(s). \end{aligned}$$

Rather than approximating the action-value function as in the vanilla method, we approximate state-value function $V_\pi(s)$ using a neural network $V(s;w)$. The objective for the state-value network is still the (squared) TD difference:

$$\delta_t = r_t + \gamma V(s_{t+1};w) - V(s_t;w).$$

The estimator of the gradient $\nabla_\theta J(\theta)$ is

$$\begin{aligned} &(\nabla_\theta \log \pi(a|s;\theta))(Q_{\pi(a|s;\theta)}(s,a) - V_{\pi(a|s;\theta)}(s)) \\ :=&(\nabla_\theta \log \pi(a|s;\theta))A_{\pi(a|s;\theta)}(s,a), \end{aligned}$$

where

$$A_{\pi(a|s;\theta)}(s,a) := Q_{\pi(a|s;\theta)}(s,a) - V_{\pi(a|s;\theta)}(s)$$

is the **advantage function**.
One remaining problem: how to compute $A_{\pi_\theta}(s_t,a_t)$ at time t? We use an estima-

tor:

$$\hat{\delta}_t := r_t + \gamma V_{\pi_\theta}(s_{t+1}) - V_{\pi_\theta}(s_t).$$

We have

$$\begin{aligned}\mathbb{E}\left[\hat{\delta}_t | s_t, a_t\right] &= \mathbb{E}\left[r_t + \gamma V_{\pi_\theta}(s_{t+1}) | s_t, a_t\right] - V_{\pi_\theta}(s_t) \\ &= Q_{\pi_\theta}(s_t, a_t) - V_{\pi_\theta}(s_t) \\ &= A_{\pi_\theta}(s_t, a_t).\end{aligned}$$

Hence, if $V(s; w) \approx V_{\pi_\theta}(s)$, then $\delta_t \approx \hat{\delta}_t$ and we use the estimator

$$(\nabla_\theta \log \pi(a_t | s_t; \theta))\delta_t$$

for the gradient $\nabla_\theta J(\theta)$.

Therefore, the algorithm (for one step) is:
(1) At time t and state s_t, sample a_t based on $\pi(a_t | s_t; \theta_t)$;
(2) Perform a_t to get s_{t+1} and reward r_t;
(3) Sample a_{t+1} based on s_{t+1} and $\pi(a_t | s_t; \theta_t)$;
(4) Compute the TD difference

$$\delta_t = r_t + \gamma V(s_{t+1}; w_t) - V(s_t; w_t)$$

and update the state value network:

$$w_{t+1} = w_t + \alpha_w \delta_t \nabla_w V(s_t; w_t);$$

(5) Policy update:

$$\theta_{t+1} = \theta_t + \alpha_\theta (\nabla_\theta \log \pi(a_t | s_t; \theta_t))\delta_t.$$